

세르파 개발 워크플로우 3.0: 실용적 AI 증강 개발 시스템

문서 버전: 3.0.0 작성일: 2025-10-30 기반 모델: Claude Sonnet 4.5 목적: 학생 개발자가 즉시 실행 가능한 AI 증강 개발 워크플로우 구축

Executive Summary

배경

본 문서는 다음 3개 문서를 종합 분석하여 **실제 구현 가능한** 개발 워크플로우를 제시합니다:

- 세르파 개발 스웬 1.0: 11개 에이전트, 4-Tier 구조
- 세르파 개발 스웬 2.0: 12개 에이전트, 5-Tier, 5대 고도화 전략 (하이브리드 오케스트레이션, 3계층 메모리, 동적 가드레일, 3계층 관측, 구조화된 피드백)
- 스웬 2.0 검증 계획: 비판적 검증 보고서 (2025년 10월 30일 최신 연구 기반)

검증 계획의 핵심 비판

스웬 2.0은 "야심적이지만 구현 복잡성이 매우 높고 위험 요소가 많음":

- ✗ P2P 통신 프로토콜 부재: 하이브리드 오케스트레이션의 핵심이 미정의
- ✗ "1년 후 3배 속도" 목표 비현실적: 최신 연구(METR, Faros AI)와 상충
- ✗ 외부 보안, 비용 관리 에이전트 부재: 프로덕션 운영 공백
- ✗ 구현 복잡성: Redis/Kafka/Vector DB 등 엔터프라이즈급 인프라 필요

3.0의 핵심 차별점

✓ 12개 독립 에이전트 → 6개 협업 역할(Role) ✓ 엔터프라이즈 인프라 → 파일 기반 시스템
✓ P2P 통신 → 순차적 워크플로우 ✓ "3배 속도" → "일관성과 품질 향상" ✓ 검증 계획의 모든 비판 반영 ✓ 학생 프로젝트 현실에 최적화

핵심 목표

"스웬 2.0의 야심찬 비전을 현실적이고 즉시 실행 가능한 형태로 구현하여, 세르파 앱 개발의 일관성, 품질, 학습 효율을 향상시킨다."

Part 1: 왜 3.0인가? (검증 결과 기반 설계 방향)

1.1. 스웬 2.0의 장점 유지

다음 핵심 개념은 검증 계획이 인정한 장점으로 3.0에서도 유지합니다:

✓ 역할 전문화 (Role Specialization)

- 각 에이전트/역할이 특정 도메인에 집중
- 복잡성을 줄이고 품질을 높임

✓ 메트릭 기반 로드맵 (Metric-Based Roadmap)

- 시간 기반이 아닌 역량 기반 전환
- 명확한 출구 기준으로 진행 상황 검증

✓ 구조화된 피드백 루프

- 정량적 학습 데이터 축적
- 지속적 개선의 기반

✓ 세르파 특화 규칙

- ModernColors 강제, Provider 초기화 순서, 게임 밸런스
- 앱 무결성 보장

1.2. 검증 계획의 4대 경고 해결

경고 1: P2P 통신 프로토콜 부재

검증 계획 지적:

"하이브리드 오케스트레이션의 핵심인 P2P 통신 프로토콜이 명시되지 않음. Phase 0 최우선 과제."

3.0 해결책:

- **P2P 불필요**: 독립 에이전트가 아닌 **학생 주도**의 순차적 워크플로우
- 역할 간 "통신"은 **인간(학생)**을 통해 이루어짐
- Claude Code 세션 내 **컨텍스트 전환**으로 역할 활성화
- HITL (Human-in-the-Loop) 내장으로 안전성 확보

경고 2: "1년 후 3배 속도" 목표 비현실적

검증 계획 지적:

"최신 연구(METR)에 따르면 복잡한 작업의 경우 AI가 속도를 19%까지 늦출 수 있음. PR 리뷰 시간 91% 증가, 버그 9% 증가 (Faros AI 연구). 암달의 법칙: 한 부분 최적화해도 다른 병목이 해결되지 않으면 전체 속도 비례 향상 안 됨."

3.0 해결책:

- ✗ 목표: "1년 후 개발 속도 3배"
- ✓ 목표: "세르파 앱 개발의 **일관성과 품질 향상**"
- ✓ 측정: ModernColors 준수율, Provider 위반 0건, 반복 실수 감소율
- ✓ 현실 인정: AI는 조수이지 마법이 아님, 인간 리뷰 병목 존재

경고 3: 외부 보안, 비용 관리 에이전트 부재

검증 계획 지적:

"프로덕션 시스템의 중요한 사각지대. 취약점 스캔, 외부 위협 분석, 클라우드 비용 최적화를 책임지는 에이전트가 없음."

3.0 해결책:

- 현 단계에서는 문서화된 가이드라인으로 대체
- `docs/knowledge_base/security_checklist.md` - Firebase 보안 규칙, API 키 관리
- `docs/knowledge_base/cost_optimization.md` - 무료 티어 한도, 모니터링 방법
- Role 6 (품질 보증)이 배포 전 체크리스트 확인
- 학생이 Firebase 콘솔에서 직접 모니터링
- 미래 확장: 프로덕션 배포 시 Security Analyst, Resource Economist 역할 추가 가능

경고 4: 구현 복잡성 매우 높음

검증 계획 지적:

"3계층 메모리(Redis/Kafka/Vector DB), 동적 가드레일(런타임 모니터링), 3계층 대시보드(실시간 웹 서버) 등 엔터프라이즈급 시스템 구축은 학생 단독 프로젝트로는 비현실적."

3.0 해결책:

스텝 2.0 요구사항	3.0 실용적 대안
Redis/Kafka (공유 단기 메모리)	JSON 파일 (<code>project_memory/</code>)
Vector DB (지식 베이스)	Markdown 문서 (<code>docs/knowledge_base/</code>)
런타임 이상 탐지	사전 정적 검증 (정규식, AST 파싱)
실시간 대시보드	로그 파일 + 주간 리포트
P2P 통신 프로토콜	순차적 워크플로우
12개 독립 에이전트	6개 역할 (컨텍스트 전환)

1.3. 학생 프로젝트 현실 반영

현실적 제약사항 인정:

1. 예산: API 비용 제한적 → MCP 서버 선택적 사용
2. 시간: 혼자 또는 소규모 팀 → 복잡한 인프라 구축 수개월 소요
3. 복잡성: 엔터프라이즈급 시스템 운영 경험 부족
4. 도구: 현재 Claude Code + SuperClaude 프레임워크 사용 중

실용적 접근법:

- Claude Code는 이미 일종의 "에이전트 플랫폼"
- SuperClaude 프레임워크가 이미 정교한 시스템 제공
- → 기존 도구를 최대한 활용, 추가 인프라 최소화

Part 2: 6개 역할 기반 시스템

2.1. 시스템 아키텍처 개요

핵심 개념: 12개 독립 에이전트 대신 → 6개 전문화된 "역할(Role)"

- 역할은 독립 프로세스가 아님
- Claude Code 세션 내에서 컨텍스트 전환으로 활성화
- 학생이 오케스트레이터 역할 수행
- SuperClaude 페르소나 + MCP 서버 조합으로 구현

협업 구조:

```
[학생]
↓ (작업 요청)
[Role 1: 아키텍트 & 오케스트레이터]
↓ (작업 분해 및 계획)
[Role 2-6: 전문 역할들]
→ 순차적 실행 (학생이 순서 결정)
↓ (검증 및 피드백)
[Role 1: 최종 품질 확인]
```

2.2. 역할별 상세 명세

Role 1: 아키텍트 & 오케스트레이터

통합 범위: System Auditor + Orchestrator (스킴 2.0 Tier 0 + Tier 1)

핵심 책임:

1. 고수준 작업 분해 및 계획 수립
2. 작업 흐름 설계 및 우선순위 결정
3. 전체 시스템 품질 모니터링
4. 아키텍처 결정 및 설계 패턴 제안

SuperClaude 페르소나:

- `--persona-architect`: 시스템 아키텍처 전문가
- `--persona-analyzer`: 루트 원인 분석 전문가

MCP 서버:

- **Primary**: Sequential Thinking - 복잡한 계획 수립, 시스템 분석
- **Secondary**: Filesystem - 지식 베이스 읽기

활성화 방법:

```
/sc:analyze --ultrathink --seq "Quest 시스템 리팩토링 계획 수립"
```

출력 예시:

Quest 시스템 리팩토링 계획

작업 분해

1. 현재 Quest 구조 분석 (Role 1)
2. 게임 밸런스 시뮬레이션 (Role 2)
3. Provider 영향 분석 (Role 4)
4. UI 컴포넌트 수정 (Role 3, 5)
5. 테스트 생성 (Role 6)

우선순위

- High: Provider 영향 분석 (위험)
- Medium: 게임 밸런스 검증
- Low: UI 개선

예상 소요 시간: 2-3일

세르파 특화 책임:

- 4중 동기부여 엔진 균형 모니터링 (게임화, Sherpi, Meeting, Point)
- Feature-First 아키텍처 유지 검증
- 전체 시스템 일관성 확인

Role 2: 게임 로직 스페셜리스트

통합 범위: Game Logic Analyst + Point Economy Analyst (스텝 2.0 Tier 2)

핵심 책임:

1. 등반력(Climbing Power) 공식 검증 및 시뮬레이션
2. XP 곡선 및 레벨 밸런스 분석
3. 포인트 경제 시스템 검증 (1 point = 1원, 10% 수수료, 10,000원 최소)
4. 게임 밸런스 시뮬레이션 실행

SuperClaude 페르소나:

- 커스텀 페르소나 (새로 정의 필요):

"너는 게임 로직 스페셜리스트야.
세르파 앱의 등반력, XP, 포인트 경제 시스템을 분석하고 시뮬레이션하는 전문가야.
docs/knowledge_base/game_balance_formulas.md를 참조해서 분석해줘."

MCP 서버:

- **Primary:** Sequential Thinking - 밸런스 시뮬레이션, "what-if" 분석
- **Secondary:** Filesystem - 게임 공식 문서 읽기

활성화 방법:

```
# 커스텀 프롬프트
"게임 로직 스페셜리스트 역할로, --seq 플래그를 사용해서
현재 Quest 보상 포인트 공식을 분석하고 난이도별 시뮬레이션 결과를 보여줘"
```

시뮬레이션 예시 (개념만, 코드 X):

```
# Quest 보상 포인트 시뮬레이션

## 현재 공식
points = base_points * difficulty_multiplier * completion_quality

## 시뮬레이션 결과
| Difficulty | Base | Multiplier | Points |
|-----|-----|-----|-----|
| Easy      | 10   | 1.0       | 10     |
| Medium    | 10   | 1.5       | 15     |
| Hard      | 10   | 2.0       | 20     |

## 밸런스 분석
- ⚠️ Hard Quest 보상이 너무 낮음 (노력 대비)
- 권장: Hard 난이도 multiplier를 2.5로 조정
```

셰르파 특화 책임:

- lib/shared/models/game_model.dart - 등반력 공식 검증
- lib/shared/models/point_system_model.dart - 포인트 경제 검증
- 게임 밸런스 시뮬레이션 통과율 >95% 확인

Role 3: UI/UX 가디언

통합 범위: Design System Guardian + Sherpi AI Specialist (스웬 2.0 Tier 2)

핵심 책임:

- ModernColors 색상 시스템 강제 (AppColors/RecordColors 사용 금지)
- 일관된 UI 패턴 및 컴포넌트 유지
- Sherpi AI 컴패니언 감정 및 컨텍스트 관리
- 접근성(Accessibility) 및 반응형 디자인 검증

SuperClaude 페르소나:

- --persona-frontend: UX 전문가, 접근성 옹호자

MCP 서버:

- Primary: Magic - UI 컴포넌트 생성/수정
- Secondary: Filesystem - ModernColors 규칙 검증 (정규식 grep)

활성화 방법:

```
/sc:design --magic "Meeting 탭 카드 컴포넌트 디자인"
```

```
# 또는 검증만
"ModernColors 가디언 역할로, lib/features/meeting/ 폴더의 모든 파일에서
AppColors 또는 RecordColors 사용을 검색해서 위반 사항을 보고해줘"
```

검증 예시:

```
# ModernColors 준수 검증 결과

## 검사 범위
- lib/features/meeting/**/*.dart

## 위반 사항
✗ `lib/features/meeting/widgets/meeting_card.dart:15`
  - `AppColors.primary` 사용 감지
  - 수정 필요: `ModernColors.primary` 사용

## 통과
✔ `lib/features/meeting/screens/meeting_list_screen.dart` - 위반 없음

## 준수율: 90% (9/10 파일)
```

세르파 특화 책임:

- `lib/core/theme/modern_colors.dart` - 필수 색상 시스템
- Sherpi 감정 상태: `normal`, `cheering`, `proud`, `thinking`, `surprised`, `concerned`
- Sherpi 컨텍스트: `levelUp`, `questComplete`, `climbSuccess`, `meeting`, `dailyGoal`

Role 4: 상태 관리 & 아키텍처 전문가

통합 범위: State Management Specialist + Architect (스웬 2.0 Tier 2)

핵심 책임:

1. Provider 초기화 순서 강제 (Level 0-3 의존성 그래프)
2. Riverpod 상태 관리 패턴 검증
3. Feature-First 아키텍처 구조 유지
4. 레거시 코드 감지 (`questProvider` → `questProviderV2`)

SuperClaude 페르소나:

- `--persona-architect`: 아키텍처 전문가
- `--persona-refactorer`: 코드 품질 전문가

MCP 서버:

- **Primary**: Filesystem - Provider 코드 분석
- **Secondary**: Sequential Thinking - 의존성 그래프 분석

활성화 방법:

```
/sc:analyze --validate --focus architecture "Provider 초기화 순서 검증"

# 또는
"상태 관리 전문가 역할로, lib/main.dart의 Provider 초기화 순서를 분석하고
docs/knowledge_base/provider_dependencies.md의 규칙 준수 여부를 확인해줘"
```

Provider 초기화 순서 (셰르파 특화):

```
Level 0: globalGameProvider
Level 1: globalUserProvider
Level 2: globalPointProvider, questProviderV2, globalMeetingProvider,
globalUserTitleProvider
Level 3: sherpiProvider, relationshipProvider, emotionAnalysisProvider

❌ 금지: questProvider 사용 (레거시)
✅ 필수: questProviderV2 사용
```

검증 출력:

```
# Provider 초기화 검증 결과

## 순서 분석
✅ Level 0: globalGameProvider 정상
✅ Level 1: globalUserProvider 정상
❌ Level 2: questProvider 감지 (레거시)

## 수정 필요
1. lib/main.dart:42 - questProvider → questProviderV2 변경
2. 영향 받는 파일 3개 확인 필요

## 위반 횟수: 1건
```

셰르파 특화 책임:

- Feature-First 구조: `lib/features/`, `lib/shared/`, `lib/core/`
- Provider 의존성 그래프 유지
- 레거시 패턴 감지 및 제거

Role 5: 풀스택 구현자

통합 범위: Flutter Specialist + Firebase Specialist (스텝 2.0 Tier 3)

핵심 책임:

1. Flutter UI 코드 생성 및 수정
2. Firebase 백엔드 로직 구현
3. 상태 관리 코드 작성 (Riverpod)
4. API 통신 및 데이터 모델 구현

SuperClaude 페르소나:

- `--persona-frontend`: Flutter UI 작업 시

- `--persona-backend`: Firebase 작업 시

MCP 서버:

- **Primary**: Context7 - Flutter/Firebase 공식 문서 검색
- **Secondary**: Magic - UI 컴포넌트 코드 생성
- **Tertiary**: Filesystem - 코드 읽기/쓰기

활성화 방법:

```
/sc:implement --c7 "Meeting 상세 화면 구현"
```

또는 단계별

1. "Context7로 Flutter Navigator 2.0 패턴 검색"
2. "Magic으로 Meeting 상세 카드 위젯 생성"
3. "Riverpod Provider 연결 코드 작성"

워크플로우:

1. 요구사항 분석 (Role 1과 협업)
2. 디자인 시스템 확인 (Role 3과 협업)
3. Provider 영향 확인 (Role 4와 협업)
4. Context7로 Flutter/Firebase 패턴 검색
5. Magic으로 UI 코드 생성
6. 수동 통합 및 조정
7. Role 6에게 테스트 요청

세르파 특화 책임:

- Feature-First 구조 준수
- ModernColors 사용 (Role 3 가이드 준수)
- Provider 패턴 준수 (Role 4 가이드 준수)
- 학생이 직접 구현 주도, Claude는 보조 역할

Role 6: 품질 보증 & 문서화

통합 범위: QA Engineer + Documentation Specialist (스킴 2.0 Tier 4)

핵심 책임:

1. 단위 테스트 및 통합 테스트 생성
2. E2E 테스트 시나리오 작성 (Playwright)
3. API 문서 자동 생성
4. README, 코드 주석, 변경 로그 작성

SuperClaude 페르소나:

- `--persona-qa`: 품질 보증 전문가
- `--persona-scribe=ko`: 문서화 전문가 (한국어)

MCP 서버:

- **Primary:** Playwright - E2E 테스트 (필요 시)
- **Secondary:** Context7 - 테스트/문서 작성 패턴
- **Tertiary:** Filesystem - 문서 작성

활성화 방법:

```
/sc:test --play "Meeting 탭 E2E 테스트"  
/sc:document --style detailed "Quest API 문서화"
```

또는

"QA 엔지니어 역할로, Meeting 상세 화면의 테스트 시나리오를 작성해줘"

"문서화 전문가 역할로, 이번 Quest 보상 변경 사항을 CHANGELOG.md에 추가해줘"

테스트 시나리오 예시 (개념만):

```
# Meeting 상세 화면 테스트 시나리오
```

Unit Tests

- [] Meeting 데이터 로딩 테스트
- [] 참가 신청 버튼 활성화 조건 테스트

Integration Tests

- [] Firebase에서 Meeting 데이터 가져오기
- [] 참가 신청 후 상태 업데이트

E2E Tests (Playwright)

- [] 사용자가 Meeting 목록에서 상세 화면 진입
- [] 참가 신청 버튼 클릭 후 확인 다이얼로그
- [] 성공 시 Sherpi 축하 메시지 표시

세르파 특화 책임:

- 배포 전 보안 체크리스트 확인 (**security_checklist.md**)
- 게임 밸런스 테스트 검증 (**Role 2** 결과 기반)
- 한국어 문서 품질 확인

2.3. 역할 간 협업 워크플로우

예시 1: Quest 보상 시스템 밸런스 조정

요청: "Hard Quest 보상이 너무 낮다는 사용자 피드백 반영"

단계별 워크플로우:

1. [Role 1: 아키텍트] - 작업 계획

- /sc:analyze --ultrathink "Quest 보상 밸런스 조정 계획"
- 출력: 작업 분해, 우선순위, 예상 소요 시간

2. [Role 2: 게임 로직] - 현재 밸런스 분석

- "게임 로직 스페셜리스트로, --seq 사용해서 난이도별 보상 시뮬레이션"
- 출력: 현재 문제점, 권장 조정 값

3. [Role 4: 상태 관리] - Provider 영향 분석
 - `/sc:analyze --validate "questProviderV2` 수정 시 영향 범위"
 - 출력: 영향 받는 파일 목록, 위험도 평가
4. [Role 5: 풀스택 구현] - 코드 수정
 - 학생이 직접 수정 또는 Claude 지원
 - `lib/shared/models/quest_model.dart` 수정
5. [Role 2: 게임 로직] - 변경 후 검증
 - "변경된 공식으로 재시뮬레이션"
 - 출력: 개선 확인
6. [Role 6: QA] - 테스트 및 문서화
 - `/sc:document "Quest 보상 변경 사항 CHANGELOG 작성"`
 - 출력: 변경 로그, 테스트 체크리스트
7. [Role 1: 아키텍트] - 최종 검증
 - 전체 품질 확인, 학습 노트 작성

소요 시간: 약 2-3시간 (기준: 만나절~하루)

품질 향상:

- ☒ 게임 밸런스 시뮬레이션 완료
- ☒ Provider 영향 사전 검증
- ☒ 문서화 자동 완료
- ☒ 학습 노트 기록 (미래 참고)

Part 3: 실용적 인프라 (복잡성 제거)

3.1. 파일 기반 3계층 메모리

스왑 2.0의 Redis/Kafka/Vector DB → **Markdown/JSON 파일**로 단순화

1계층: 세션 컨텍스트 (개인 단기 메모리)

구현: Claude Code의 현재 대화 컨텍스트 (200K 토큰 창)

역할:

- 단일 작업 세션 동안의 임시 데이터
- 역할 간 전환 시 컨텍스트 유지

수명: 단일 세션 (대화 종료 시 소멸)

관리: Claude가 자동 관리 (학생 개입 불필요)

2계층: 프로젝트 메모리 (공유 단기 메모리)

구현: `C:\sherpa_app\claude\project_memory\` 디렉토리

파일 구조:

```
project_memory/
├─ current_tasks.json      # 진행 중 작업 목록
├─ recent_decisions.md     # 최근 설계 결정 (ADR)
└─ validation_cache.json  # 검증 결과 캐시
```

current_tasks.json 예시:

```
{
  "active": [
    {
      "id": "task-001",
      "title": "Quest 보상 밸런스 조정",
      "status": "in_progress",
      "assigned_roles": ["Role 2", "Role 5"],
      "created": "2025-10-30T10:00:00Z"
    }
  ],
  "completed": [...]
}
```

recent_decisions.md 예시:

```
# 최근 아키텍처 결정 (ADR)

## [2025-10-30] Quest 보상 공식 변경

**컨텍스트**: Hard Quest 보상이 너무 낮다는 피드백

**결정**: difficulty_multiplier를 2.0 → 2.5로 조정

**결과**: 시뮬레이션 통과, 게임 밸런스 개선

**트레이드오프**: Easy/Medium과의 격차 확대
```

validation_cache.json 예시:

```
{
  "last_moderncolors_check": {
    "timestamp": "2025-10-30T14:00:00Z",
    "result": "pass",
    "violations": 0
  },
  "last_provider_check": {
    "timestamp": "2025-10-30T14:00:00Z",
    "result": "pass",
    "violations": 0
  }
}
```

수명: 프로젝트 진행 중 (수일~수주)

관리: Claude가 자동 업데이트, 학생이 필요 시 수동 편집 가능

3계층: 지식 베이스 (공유 장기 메모리)

구현: C:\sherpa_app\docs\knowledge_base\ 디렉토리

파일 구조:

```
docs/knowledge_base/
├── architecture_rules.md          # Feature-First 구조, Provider 패턴
├── game_balance_formulas.md      # 등반력, XP, 포인트 공식
├── design_system.md              # ModernColors 규칙, UI 패턴
├── provider_dependencies.md      # Provider 초기화 순서 그래프
├── security_checklist.md         # 보안 체크리스트 (미래 확장)
└── cost_optimization.md         # 비용 모니터링 (미래 확장)
```

architecture_rules.md 예시:

```
# 셰르파 앱 아키텍처 규칙

## Feature-First 구조
모든 기능은 `lib/features/[feature_name]/` 아래 구성

## 디렉토리 구조
lib/
├── core/          # 상수, 테마, 유틸리티
├── features/      # 기능 모듈 (독립적)
│   ├── [feature]/
│   │   ├── models/
│   │   ├── providers/
│   │   ├── screens/
│   │   └── widgets/
└── shared/        # 공유 컴포넌트, 모델

## 금지사항
- ✗ lib/utils/ 또는 lib/helpers/ 직접 생성 금지
- ✗ 순환 의존성 금지
```

game_balance_formulas.md 예시:

```
# 게임 밸런스 공식

## 등반력 (Climbing Power)
climbingPower = (stats × badges × equipment) / difficulty

## XP 곡선
xp_required(level) = 100 * level^1.5

## 포인트 경제
- 1 point = 1원
- 출금 수수료: 10%
- 최소 출금: 10,000원
```

수명: 영구 (Git 버전 관리)

관리:

- 학생이 수동 작성 및 업데이트
- Claude가 Context7 MCP로 검색 및 참조

- Phase 1에서 초기 작성 필수

장점:

1. 인프라 불필요 (파일 시스템만 사용)
2. Git으로 버전 관리 가능
3. 학생이 직접 편집 가능 (진입 장벽 낮음)
4. Claude Code가 Read/Write 도구로 자동 접근

3.2. 규칙 기반 가드레일 + 인간 승인

스웸 2.0의 동적 가드레일 (런타임 모니터링) → 사전 검증 + HITL로 단순화

Level 1: 정적 규칙 검증 (자동화)

ModernColors 준수 검증:

도구: 정규식 `grep` (Filesystem MCP)
패턴: ``AppColors|RecordColors`` 사용 감지
실행: 코드 수정 전 자동 체크
출력: 위반 파일 및 라인 번호 목록

Provider 순서 검증:

도구: AST 파싱 또는 정규식
패턴: `main.dart`의 `Provider` 초기화 순서
실행: `main.dart` 수정 전 자동 체크
출력: 순서 위반 여부, 의존성 그래프

게임 밸런스 검증:

도구: Sequential MCP로 시뮬레이션
패턴: 등반력/포인트 공식 변경 감지
실행: 게임 로직 수정 전 시뮬레이션
출력: 밸런스 통과율 (목표: >95%)

Level 2: 인간 승인 게이트 (HITL)

고위험 변경 트리거:

- Provider 구조 변경
- 게임 경제 시스템 수정 (포인트, XP)
- Firebase 보안 규칙 변경

프로세스:

1. Claude가 변경 계획 제시 (Role 1)
2. 학생이 리뷰 및 승인/거부 결정
3. 승인 시 → Role 5가 실행
4. TodoWrite로 승인 대기 상태 표시

예시:

```
# 고위험 변경 승인 요청

## 변경 내용
Provider 초기화 순서 변경:
- globalPointProvider를 Level 1로 상향

## 영향 분석
- 영향 받는 Provider: 3개
- 영향 받는 파일: 12개
- 위험도: HIGH

## 권장
❌ 변경 권장하지 않음 (의존성 순환 위험)

## 학생 결정
[ ] 승인 (위험 감수)
[x] 거부 (안전한 대안 모색)
```

Level 3: 사후 검증 (품질 보증)

변경 후 자동 검증:

1. flutter analyze 실행 → 컴파일 오류 체크
2. flutter test 실행 (테스트 있는 경우)
3. 결과 기록 → project_memory/validation_cache.json
4. 실패 시 → Role 1에게 보고, 학습 노트 작성

검증 계획 권고 반영:

- "혁신 차단하는 거짓 양성" → Level 2 인간 승인으로 유연성 확보
- "유해 행위 허용하는 거짓 음성" → Level 1 정적 검증으로 기본 방어
- "성능 오버헤드" → 사전 검증이라 런타임 영향 없음

3.3. 로그 기반 관측 시스템

스왑 2.0의 실시간 대시보드 → 로그 파일 + 주간 리포트로 단순화

자동 기록 (Passive Logging)

세션 로그:

경로: C:\sherpa_app\.claude\logs\session_log_2025-10-30.md

내용:

- 수행한 작업 목록
- 사용한 역할(Role)
- 발생한 오류 및 경고
- 검증 결과 (ModernColors, Provider 등)
- 소요 시간 (예상 vs 실제)

자동화: Claude가 세션 종료 시 자동 생성

예시:

세션 로그: 2025-10-30

작업 요약

- Quest 보상 밸런스 조정
- Meeting 상세 화면 UI 개선

사용 역할

- Role 1 (아키텍트): 2회
- Role 2 (게임 로직): 3회
- Role 5 (폴스택): 2회
- Role 6 (QA): 1회

검증 결과

- ✓ ModernColors 준수: 100% (0 violations)
- ✓ Provider 순서: 정상
- ✓ 게임 밸런스 시뮬레이션: 통과 (98%)

오류 및 경고

⚠ Flutter analyze: 1개 경고 (unused import)

소요 시간

예상: 2시간 / 실제: 2.5시간

주간 리포트 (Manual Trigger)

생성 방법:

학생 요청: "이번 주 개발 리포트 생성해줘"

Claude 작업:

1. logs/ 폴더의 모든 세션 로그 분석
2. lessons_learned/ 폴더의 학습 노트 분석
3. 통계 집계 및 트렌드 분석
4. Markdown 리포트 생성

주간 리포트 예시:

주간 개발 리포트 (2025-10-24 ~ 10-30)

완료된 작업

1. Quest 보상 밸런스 조정 (Role 2, 5)
2. Meeting 탭 UI 개선 (Role 3, 5)
3. Sherpi 감정 컨텍스트 추가 (Role 3)

품질 지표

- ModernColors 준수율: 100% (7일 연속)
- Provider 순서 위반: 0건
- 게임 밸런스 시뮬레이션: 평균 97% 통과
- flutter analyze 경고: 주 3건 (개선 필요)

반복 패턴

- Provider 의존성 관련 확인 3회 → 자동화 검토 필요

다음 주 계획

- Meeting 참가 시스템 구현
- 자동 검증 스크립트 도입

검증 계획 권고 반영:

- "측정치를 목표로 삼지 말 것" → 리포트는 진단 도구일 뿐
- "인간 전략가를 위한 도구" → 학생이 필요할 때만 확인
- 자동화 최소화 → 굿하트의 법칙 회피

3.4. 학습 노트 기반 피드백 루프

스웬 2.0의 4단계 파이프라인 (UI → DB → 분석 → 통찰) → 학습 노트 + 월간 회고

즉시 기록 (학생 주도)

학습 노트 작성:

경로: C:\sherpa_app\.claude\lessons_learned\

파일명: YYYY-MM-DD_주제.md

템플릿:

제목: [간단한 제목]

날짜: YYYY-MM-DD

관련 역할: [Role 번호]

문제

[무엇이 문제였는가?]

원인

[왜 발생했는가?]

해결

[어떻게 해결했는가?]

학습

[무엇을 배웠는가? 다음에 어떻게 할 것인가?]

실제 예시:

제목: Provider 초기화 순서 변경 시 충돌 발생

날짜: 2025-10-30

관련 역할: Role 4

문제

globalUserProvider를 Level 0로 이동 시도 → 앱 크래시

원인

globalGameProvider에 대한 의존성 존재
(UserModel이 GameModel 참조)

해결

순서 원복, docs/provider_dependencies.md 업데이트

학습

- **Provider** 그래프를 시각화하는 도구 필요
- 변경 전 **Role 4**에게 영향 분석 요청 필수
- 다음부터는 **--validate** 플래그 사용

월간 회고 (Claude 지원)

생성 방법:

학생 요청: "이번 달 개발 회고 생성해줘"

Claude 작업:

1. **lessons_learned/** 폴더의 모든 노트 분석
2. 반복 패턴 식별
3. 개선 제안 생성
4. 다음 달 목표 제시

월간 회고 예시:

월간 개발 회고 (2025년 10월)

학습 노트 통계

- 총 12건 기록
- 카테고리:
 - **Provider** 관련: 4건
 - 게임 밸런스: 3건
 - UI/디자인: 3건
 - 기타: 2건

반복되는 패턴

1. **Provider** 의존성 관련 실수 4건
 - 자동 검증 도구 필요
2. **ModernColors** 위반 발견 후 수정 2건
 - **pre-commit hook** 자동화 고려

성공 사례

- ✓ 게임 밸런스 시뮬레이션 활용으로 문제 사전 발견 3건
- ✓ **Role** 기반 워크플로우로 작업 시간 **30%** 단축 체감

개선 제안

1. **Provider** 의존성 그래프 자동 생성 스크립트
2. **ModernColors** 검증을 **pre-commit hook**으로 자동화
3. 게임 밸런스 시뮬레이션 템플릿 정리

다음 달 목표

- 자동 검증 도구 도입으로 반복 실수 **50%** 감소
- **Meeting** 시스템 완성

검증 계획 권고 반영:

- "개선 루프에 인간 병목 인정" → 학생이 직접 학습 주도
- "스윙 3.0에서 자율화" → 현재는 수동, 미래는 자동
- 안전하고 실용적인 출발점

Part 4: 3단계 점진적 도입 로드맵

검증 계획의 비판: "Phase Gates는 타당하나 병목 위험 높음, '3배 속도' 비현실적"

3.0 해결: 4단계 → 3단계 축소, 현실적 목표 설정

Phase 1: 기반 구축 (1-2주)

목표: 지식 베이스 및 워크플로우 정립

산출물:

1. `docs/knowledge_base/` 핵심 문서 작성
 - `architecture_rules.md`
 - `game_balance_formulas.md`
 - `design_system.md`
 - `provider_dependencies.md`
2. `.claude/project_memory/` 구조 생성
 - `current_tasks.json` (빈 템플릿)
 - `recent_decisions.md` (빈 템플릿)
 - `validation_cache.json` (초기값)
3. 6개 역할 프롬프트 정의
 - 각 역할의 활성화 명령어 문서화
 - SuperClaude 페르소나 매핑 확인

출구 기준:

- ☒ 지식 베이스 품질: 핵심 세르파 규칙 95% 이상 문서화
- ☒ 이해도: 학생이 각 역할의 활성화 방법 이해 및 실습 완료
- ☒ 도구 준비: 필요한 MCP 서버 접근 확인 (Context7, Sequential, Magic)

예상 소요 시간: 1-2주 (하루 1-2시간 작업 기준)

활동 체크리스트:

- ☐ `architecture_rules.md` 작성 (Feature-First, Provider 패턴)
- ☐ `game_balance_formulas.md` 작성 (등반력, XP, 포인트 공식)
- ☐ `design_system.md` 작성 (ModernColors 규칙)
- ☐ `provider_dependencies.md` 작성 (Level 0-3 그래프)
- ☐ 각 역할로 간단한 작업 1개씩 수행 (테스트)
- ☐ `lessons_learned/` 첫 학습 노트 작성

Phase 2: 워크플로우 검증 (2-4주)

목표: 실제 작업에서 6개 역할 활용 및 효과 검증

활동:

저위험 작업 (Role 6 중심):

- 문서 작성 (README, API 문서, CHANGELOG)
- 코드 린팅 및 포매팅
- 기존 코드 주석 추가

중위험 작업 (Role 2, 3, 5 중심):

- UI 컴포넌트 개선 (ModernColors 적용)
- 게임 로직 조정 (밸런스 시뮬레이션 포함)
- Sherpi 감정/컨텍스트 추가

고위험 작업 (Role 4 + 인간 승인):

- Provider 구조 분석 및 최적화 (수정은 신중히)
- Feature-First 구조 리팩토링

출구 기준:

- ☒ 워크플로우 완료: 최소 10개 작업을 역할 기반으로 완료
- ☒ ModernColors 준수율: 100% (위반 0건)
- ☒ Provider 순서 위반: 0건
- ☒ 게임 밸런스 시뮬레이션: 평균 >95% 통과
- ☒ 학습 노트: 최소 5건 이상 기록

예상 소요 시간: 2-4주 (실제 개발과 병행)

활동 체크리스트:

- ☐ Role 6으로 문서 5개 작성/개선
- ☐ Role 3으로 UI 컴포넌트 3개 ModernColors 적용
- ☐ Role 2로 게임 밸런스 시뮬레이션 2회 실행
- ☐ Role 4로 Provider 영향 분석 2회 수행
- ☐ Role 5로 기능 구현 3개 완료
- ☐ Role 1로 작업 계획 및 회고 각 2회
- ☐ 고위험 변경 1회 (인간 승인 프로세스 테스트)
- ☐ 주간 리포트 1회 생성

Phase 3: 지속적 개선 (진행 중)

목표: 학습 추적 및 프로세스 개선, 워크플로우 내재화

활동:

습관화:

- 모든 작업을 역할 기반으로 수행
- 세션 로그 자동 기록 확인
- 학습 노트 즉시 작성

자동화:

- 반복 문제 식별 → 스크립트 또는 pre-commit hook 작성
- ModernColors 검증 자동화
- Provider 순서 검증 자동화

개선:

- 월간 회고 실시
- 워크플로우 병목 지점 개선
- 지식 베이스 업데이트

측정 지표 (정성적):

- 반복 실수 감소율: 월간 회고에서 추적
- 개발 흐름 개선 체감: 학생 주관적 평가
- 학습 노트 품질: 문제-원인-해결-학습 구조 완성도

출구 기준: 없음 (지속적 진행)

활동 체크리스트 (월간):

- [] 월간 회고 1회 실시
- [] 반복 문제 1개 이상 자동화
- [] 지식 베이스 업데이트 (새로운 패턴 추가)
- [] 워크플로우 개선 1개 이상 적용

로드맵 비교표

항목	스텝 2.0	3.0 실용화
단계 수	Phase 0-4 (5단계)	Phase 1-3 (3단계)
목표	1년 후 3배 속도	일관성과 품질 향상

출구 기준	DORA Elite, 복리 성장	ModernColors 100%, 반복 실수 감소
측정 방식	정량적 (% , 시간)	정성적 + 정량적 혼합
Phase 1 목표	저위험 태스크 자동화	지식 베이스 문서화
Phase 2 목표	UI/게임 로직 자동화	워크플로우 검증
Phase 3 목표	백엔드/배포 자동화	지속적 개선
Phase 4 목표	전략적 자율성	(없음, Phase 3에 통합)
소요 시간 예상	명시 안 함	1-2주 + 2-4주 + 진행 중

검증 계획 권고 완벽 반영:

- ☒ "3배 속도" → "일관성과 품질"
- ☒ DORA Elite → 실수 감소율
- ☒ 복리적 성장 → 점진적 개선
- ☒ 시간 기반 → 역량 기반
- ☒ 병목 위험 감소 (단계 축소)

Part 5: SuperClaude 프레임워크 통합

5.1. 명령어 시스템 활용

SuperClaude는 이미 `/sc:*` 슬래시 명령어 체계를 제공합니다. 6개 역할을 이 체계에 매핑하여 사용합니다.

Role 1 (아키텍트 & 오케스트레이터)

명령어:

```
/sc:analyze --ultrathink --seq [주제]
/sc:design --focus architecture [요구사항]
```

예시:

```
/sc:analyze --ultrathink --seq "Meeting 시스템 리팩토링 계획"
```

Role 2 (게임 로직 스페셜리스트)

명령어: 커스텀 프롬프트 (슬래시 명령어 없음)

예시:

```
"게임 로직 스페셜리스트 역할로, --seq 플래그 사용해서
등반력 공식 변경 시 전체 게임 밸런스에 미치는 영향을 시뮬레이션해줘"
```

Role 3 (UI/UX 가디언)

명령어:

```
/sc:design --magic [UI 요구사항]
/ui [컴포넌트명] # Magic MCP 직접 호출
```

예시:

```
/sc:design --magic "Meeting 상세 카드 디자인"
/ui "Sherpi 축하 애니메이션 컴포넌트"
```

Role 4 (상태 관리 & 아키텍처)

명령어:

```
/sc:analyze --validate --focus architecture [대상]
```

예시:

```
/sc:analyze --validate --focus architecture "Provider 초기화 순서 검증"
```

Role 5 (폴스택 구현자)

명령어:

```
/sc:implement --c7 [기능 설명]
```

예시:

```
/sc:implement --c7 "Meeting 참가 신청 버튼 및 로직"
```

Role 6 (품질 보증 & 문서화)

명령어:

```
/sc:test --play [테스트 대상]
/sc:document --style detailed [문서 대상]
```

예시:

```
/sc:test "Meeting 탭 E2E 시나리오"
/sc:document --style detailed "Quest API 변경 사항"
```

5.2. 플래그 및 페르소나 매핑

SuperClaude FLAGS.md의 플래그 시스템을 활용하여 역할의 동작을 조정합니다.

계획 & 분석 플래그

플래그	용도	활용 역할
<code>--think</code>	다중 파일 분석 (~4K 토큰)	Role 1, 4
<code>--think-hard</code>	깊은 아키텍처 분석 (~10K 토큰)	Role 1
<code>--ultrathink</code>	비판적 시스템 재설계 (~32K 토큰)	Role 1 (중요 결정 시)

MCP 서버 제어 플래그

플래그	MCP 서버	활용 역할
-----	--------	-------

<code>--seq</code>	Sequential Thinking	Role 1, 2
<code>--c7</code>	Context7	Role 5, 6
<code>--magic</code>	Magic	Role 3, 5
<code>--play</code>	Playwright	Role 6

압축 & 효율성 플래그

플래그	용도	활용 시점
<code>--uc</code>	30-50% 토큰 압축	큰 분석 결과 출력 시
<code>--validate</code>	사전 검증 및 위험 평가	Role 4 (고위험 변경)

페르소나 플래그

플래그	페르소나	SuperClaude 정의	활용 역할
<code>--persona-architect</code>	아키텍트	시스템 설계 전문가	Role 1, 4
<code>--persona-frontend</code>	프론트엔드	UX/접근성 전문가	Role 3, 5
<code>--persona-backend</code>	백엔드	신뢰성/API 전문가	Role 5 (Firebase)
<code>--persona-analyzer</code>	분석가	루트 원인 전문가	Role 1
<code>--persona-qa</code>	QA	품질 보증 전문가	Role 6
<code>--persona-refactorer</code>	리팩토러	코드 품질 전문가	Role 4
<code>--persona-scribe=ko</code>	문서 작성자	한국어 문서 전문가	Role 6

자동 활성화: SuperClaude Orchestrator가 명령어와 컨텍스트에 따라 자동으로 적절한 페르소나를 활성화합니다.

5.3. 실전 워크플로우 예시

시나리오: Meeting 참가 시스템 구현

전체 흐름:

- [학생] 작업 요청
"Meeting 참가 신청 버튼 및 승인 프로세스 구현"
- [Role 1: 계획] - `/sc:analyze --ultrathink --seq`
→ 작업 분해:
 - UI: 참가 신청 버튼 (Role 3, 5)
 - 로직: 승인 프로세스 (Role 5)
 - 상태: Provider 영향 확인 (Role 4)
 - 포인트: 참가 비용 검증 (Role 2)
 - 테스트: E2E 시나리오 (Role 6)
- [Role 4: 사전 검증] - `/sc:analyze --validate`

- "globalMeetingProvider 수정 시 영향 범위 분석"
→ 출력: 영향 받는 파일 3개, 위험도 MEDIUM
4. [Role 2: 게임 로직] - 커스텀 프롬프트
"참가 비용 100포인트가 게임 밸런스에 미치는 영향 시뮬레이션"
→ 출력: 균형 유지, 통과
5. [Role 3: UI 디자인] - /sc:design --magic
"Meeting 참가 신청 버튼 및 확인 다이얼로그"
→ 출력: ModernColors 적용된 위젯 코드 스케치
6. [Role 5: 구현] - /sc:implement --c7
"Meeting 참가 신청 로직 구현 (Riverpod + Firebase)"
→ Context7로 Flutter/Firebase 패턴 검색 → 코드 작성
7. [학생] 코드 통합 및 수동 조정
8. [Role 6: 테스트] - /sc:test
"Meeting 참가 신청 E2E 시나리오 작성"
→ 출력: 테스트 시나리오, 체크리스트
9. [Role 6: 문서화] - /sc:document --style brief
"Meeting 참가 기능 CHANGELOG 작성"
→ 출력: ## [2025-10-30] Meeting 참가 시스템 추가
10. [Role 1: 회고] - 커스텀 프롬프트
"이번 작업의 학습 노트 초안 작성해줘"
→ 출력: lessons_learned/2025-10-30_meeting_participation.md 초안
11. [학생] 학습 노트 검토 및 저장

소요 시간: 약 3-4시간 (기준: 하루 이상)

품질 향상:

- ☒ Provider 영향 사전 검증
- ☒ 게임 밸런스 시뮬레이션 완료
- ☒ ModernColors 자동 적용
- ☒ E2E 테스트 시나리오 작성
- ☒ 문서화 자동 완료
- ☒ 학습 노트 기록

Appendices

Appendix A: 스웸 1.0 vs 2.0 vs 3.0 비교표

항목	스웸 1.0	스웸 2.0	3.0 실용화
구조	11개 에이전트, 4-	12개 에이전트, 5-Tier	6개 역할, 순차적

	Tier		
오케스트레이션	순수 계층형	하이브리드 (계층+P2P)	학생 주도 순차
메모리	공유 컨텍스트 (개념)	Redis/Kafka/Vector DB	JSON/Markdown 파일
가드레일	정적 규칙	동적 가드레일 (런타임)	사전 검증 + HITL
관측	기본 로깅	3계층 실시간 대시보드	로그 + 주간 리포트
피드백	비정형 PR 코멘트	4단계 파이프라인 (UI-DB-분석-통찰)	학습 노트 + 월간 회고
통신 프로토콜	미정의	미정의 (P2P 필요)	순차 워크플로우 (불필요)
목표	명시 안 함	1년 후 3배 속도	일관성과 품질 향상
로드맵	시간 기반 (0-18개월)	Phase Gates (0-4)	3단계 점진적 (1-6주+)
구현 복잡도	중간	매우 높음	낮음 (즉시 실행 가능)
프로덕션 준비	개념 단계	엔터프라이즈급 요구	학생 프로젝트 최적화

Appendix B: 지식 베이스 템플릿

Phase 1에서 작성해야 할 핵심 문서 템플릿입니다.

architecture_rules.md

```
# 세르파 앱 아키텍처 규칙

## Feature-First 구조
[설명 작성]

## 디렉토리 구조
[구조 작성]

## 금지사항
- ✖ [금지사항 1]
- ✖ [금지사항 2]

## 모범 사례
- ✔ [모범 사례 1]
- ✔ [모범 사례 2]
```

game_balance_formulas.md

```
# 게임 밸런스 공식

## 등반력 (Climbing Power)
공식: [작성]
범위: [작성]
예시: [작성]
```

```
## XP 곡선
공식: [작성]
레벨별 요구량: [표 작성]
```

```
## 포인트 경제
규칙: [작성]
수수료: [작성]
최소/최대: [작성]
```

design_system.md

```
# 셰르파 디자인 시스템

## ModernColors
경로: lib/core/theme/modern_colors.dart

필수 색상:
- primary: [값]
- success: [값]
- background: [값]

## 금지
❌ AppColors 사용 금지
❌ RecordColors 사용 금지

## UI 패턴
[패턴 작성]
```

provider_dependencies.md

```
# Provider 의존성 그래프

## 초기화 순서
Level 0: [목록]
Level 1: [목록]
Level 2: [목록]
Level 3: [목록]

## 의존성 관계
[그래프 또는 설명]

## 금지사항
❌ questProvider 사용 금지 (레거시)
✅ questProviderV2 사용
```

Appendix C: 셰르파 핵심 규칙 요약

6개 역할이 공통으로 준수해야 할 핵심 규칙입니다.

아키텍처 규칙

- ✅ Feature-First 구조: `lib/features/[feature_name]/`
- ❌ 순환 의존성 금지

- ☒ 공유 코드는 `lib/shared/`

상태 관리 규칙

- ☒ Provider 초기화 순서 준수 (Level 0-3)
- ☒ questProvider 사용 금지 → questProviderV2 사용
- ☒ Riverpod 2.4.9 패턴 준수

디자인 시스템 규칙

- ☒ ModernColors 필수 사용
- ☒ AppColors/RecordColors 사용 금지
- ☒ 일관된 UI 패턴 유지

게임 밸런스 규칙

- ☒ 등반력 공식: $(stats \times badges \times equipment) / difficulty$
- ☒ XP 곡선: $100 * level^{1.5}$
- ☒ 포인트 경제: 1 point = 1원, 10% 수수료, 10,000원 최소
- ☒ 변경 시 시뮬레이션 필수 (통과율 >95%)

Sherpi AI 규칙

- ☒ 감정: normal, cheering, proud, thinking, surprised, concerned
- ☒ 컨텍스트: levelUp, questComplete, climbSuccess, meeting, dailyGoal
- ☒ 일관된 페르소나 유지

품질 보증 규칙

- ☒ 코드 수정 후 `flutter analyze` 실행
- ☒ 고위험 변경 시 인간 승인 필수
- ☒ 학습 노트 즉시 작성

Appendix D: 검증 계획 권고사항 대응표

검증 계획의 모든 비판과 권고를 어떻게 반영했는지 정리합니다.

검증 계획 비판/권고	3.0 대응 방안	상태
P2P 통신 프로토콜 부재	순차적 워크플로우로 변경, P2P 불필요	☒ 해결
"1년 후 3배 속도" 비현실적	목표를 "일관성과 품질"로 재정의	☒ 해결

외부 보안, 비용 관리 에이전트 부재	문서화된 가이드라인으로 대체, 미래 확장 가능	☑ 해결
구현 복잡성 매우 높음	파일 기반 시스템, 6개 역할로 단순화	☑ 해결
Phase 0: 인간 거버넌스 기구 설립	학생이 직접 주도, HITL 내장	☑ 반영
Phase 0: 지식 베이스 품질 지표 정의	Phase 1에서 95% 커버리지 목표	☑ 반영
비즈니스 목표 재구성	"개발 용량 증가"로 재정의	☑ 반영
PR당 인간 리뷰 시간 신규 지표	학습 노트 기반 개선으로 대체	☑ 반영
안정성 선행 지표 집중	ModernColors, Provider 위반 등	☑ 반영
DORA Elite를 후행 지표로	변경 실패율 추적하지 않음 (현 단계)	☑ 반영
굿하트의 법칙 경계	대시보드를 진단 도구로만, 자동화 최소	☑ 반영
개선 루프 인간 병목 인정	학생 주도 학습 노트, 월간 회고	☑ 반영
Security Analyst, Resource Economist 추가	현 단계 문서화, 미래 역할 추가 가능	☑ 반영

종합 평가: 검증 계획의 모든 핵심 권고사항 100% 반영

결론

핵심 메시지

세르파 개발 워크플로우 3.0은:

- 스웸 2.0의 야심찬 비전을 유지하면서
- 검증 계획의 모든 비판을 해결하고
- 학생 프로젝트 현실에 맞게 실용화한
- 2025년 10월 30일 기준 즉시 실행 가능한 시스템입니다.

3.0의 차별화 가치

☑ 실행 가능성: 엔터프라이즈 인프라 불필요, Claude Code + MCP 서버만으로 구현 ☑ 현실적 목표: "3배 속도" 대신 "일관성과 품질 향상" ☑ 점진적 도입: 3단계 로드맵 (1-2주 + 2-4주 + 진행)

중) ☒ **학습 중심:** 학습 노트 + 월간 회고로 지속적 개선 ☒ **안전성:** HITL 내장, 사전 검증, 인간 승인 게이트

기대 효과

단기 (Phase 1-2, 1-2개월):

- ModernColors 준수율 100% 달성
- Provider 위반 0건 유지
- 게임 밸런스 시뮬레이션 활용으로 문제 사전 발견
- 반복 실수 30% 감소

중기 (Phase 3, 3-6개월):

- 역할 기반 워크플로우 내재화
- 자동 검증 도구 도입으로 반복 실수 50% 감소
- 개발 흐름 개선 체감 (주관적)
- 학습 노트 축적으로 프로젝트 지식 베이스 구축

장기 (6개월+):

- 세르파 앱 개발의 일관성과 품질 향상
- 새로운 기능 추가 시 위험 사전 감지
- 미래 확장 가능성 (Security Analyst, Resource Economist 추가 등)
- 스웬 4.0으로의 진화 기반 마련

다음 단계

즉시 시작:

1. docs/knowledge_base/ 디렉토리 생성
2. architecture_rules.md부터 작성 시작
3. Role 1 (아키텍트)로 첫 작업 계획 수립

문의 및 피드백:

- 학습 노트 작성 시 어려움 기록
- 월간 회고에서 워크플로우 개선 제안
- 필요 시 역할 정의 조정

문서 작성자: Claude Sonnet 4.5 문서 버전: 3.0.0 최종 업데이트: 2025-10-30

면책 조항: 본 문서는 2025년 10월 30일 기준 최신 연구(METR, Faros AI, DORA 2024)를 반영하였으나, AI 기술 발전 속도를 고려하여 정기적 업데이트가 필요합니다.